Algorítmica

Tema 4. Algoritmos basados en programación dinámica

Francisco Javier Cabrerizo Lorite

Curso académico 2024-2025

ETS de Ingenierías Informática y de Telecomunicación. Universidad de Granada

Índice

1. Introducción

2. Principio de optimalidad de Bellman

3. Características generales

4. Ejemplos de aplicación

Objetivos

- Saber identificar soluciones distintas para un problema y evaluar su bondad.
- Ser consciente de la importancia del análisis de la eficiencia de un algoritmo como paso previo a su implementación.
- Conocer la técnica de resolución de problemas de programación dinámica.
- Ser capaz de identificar las diferencias de la técnica de programación dinámica respecto a las técnicas de «divide y vencerás» y *greedy*.
- Saber identificar los problemas que cumplen el principio de optimalidad y qué se necesita para poder aplicar la técnica de programación dinámica.

Introducción

Sucesión de Fibonacci

- La ecuación de recurrencias de la sucesión de Fibonacci es:

$$F_n = \begin{cases} n & n = 0, 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}$$

- Algoritmo recursivo:

```
función fibonacci(n)
    si n ≤ 1 entonces
        devolver n
    si no
        devolver fibonacci(n−1) + fibonacci(n−2)
    fin si
fin función
```

- $T(n) \in \mathrm{O}(\Phi^n)$

Algoritmo recursivo de Fibonacci

- Es ineficiente porque se repiten muchos cálculos:

$$\begin{aligned} F_6 &= F_5 + F_4 \\ &= F_4 + F_3 + F_3 + F_2 \\ &= F_3 + F_2 + F_2 + F_1 + F_2 + F_1 + F_2 \\ &= F_2 + F_1 + F_2 + F_2 + F_1 + F_2 + F_1 + F_2 \end{aligned}$$

- Los subproblemas se solapan.

- Se podría mejorar si se almacenan los resultados y se calcula cada F_i una única vez.

Algoritmo iterativo de Fibonacci

- Se puede obtener un algoritmo más eficiente usando una tabla para almacenar los cálculos si estos se organizan de forma ordenada.

```
función fibonacci_iterativo(n)
    tabla[0..n]
    si n ≤ 1 entonces
        devolver n
    si no
        tabla[0] ← 0
        tabla[1] ← 1
        para todo i = 2, . . . , n hacer
            tabla[i] ← tabla[i−1] + tabla[i−2]
        fin para todo
        devolver tabla[n]
    fin si
fin función
```

- $T(n) \in \mathrm{O}(n)$

- El coste espacial es lineal.

Algoritmo iterativo de Fibonacci eficiente en espacio

- Como el término F_n depende exclusivamente de F_{n-1} y F_{n-2}, no se necesita almacenar toda la tabla.

```
función fibonacci_eficiente_espacio(n)
    t1 ← 1
    t2 ← 0
    si n ≤ 1 entonces
        devolver n
    si no
        para todo i = 2, . . . , n hacer
            tn ← t1 + t2
            t2 ← t1
            t1 ← tn
        fin para todo
        devolver tn
    fin si
fin función
```

- $T(n) \in \mathrm{O}(n)$

- El coste espacial es constante.

Números combinatorios

- Calcula el número de subconjuntos de tamaño k que se pueden formar con n elementos distintos:

$$\binom{n}{k} = \frac{n!}{(n-k)!k!}$$

- Se puede calcular recursivamente:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

- Algoritmo recursivo:

```
función combinatorio(n, k)
    si k = 0 || k = n entonces
        devolver 1
    si no
        devolver combinatorio(n-1,k-1) + combinatorio(n-1,k)
    fin si
fin función
```

Algoritmo recursivo de los números combinatorios

- Es ineficiente porque muchos valores $C(i, j)$, con $i < n$, $j < k$, se calculan varias veces:

$$
\begin{aligned}
C(5,3) &= C(4,2) + C(4,3) \\
&= C(3,1) + C(3,2) + C(3,2) + C(3,3) \\
&= C(2,0) + C(2,1) + C(2,1) + C(2,2) + C(2,1) + C(2,2) + 1 \\
&= 1 + C(1,0) + C(1,1) + C(1,0) + C(1,1) + 1 + C(1,0) + C(1,1) + 1 + 1 \\
&= 10
\end{aligned}
$$

Algoritmo iterativo de los números combinatorios

- Si se usa una tabla de resultados intermedios $C[i, j]$, $i = 1, \ldots, n$, $j = 0, \ldots, k$ (triángulo de Pascal) se obtiene un resultado más eficiente.

- Se rellena la tabla línea por línea:

	0	1	2	3	4	5	$\cdots$
0	1						
1	1	1					
2	1	2	1				
3	1	3	3	1			
4	1	4	6	4	1		
5	1	5	10	10	5	1	
$\vdots$			$\vdots$				$\ddots$

Algoritmo iterativo de los números combinatorios

```
función combinatorio_iterativo(n, k)
    tabla[0..n][0..k]
    para todo i = 0, . . . , n hacer
        para todo j = 0, . . . , k hacer
            si j = 0 || j = i entonces
                tabla[i][j] ← 1
            si no
                tabla[i][j] ← tabla[i−1][j−1] + tabla[i−1][j]
            fin si
        fin para todo
    fin para todo
    devolver tabla[n][k]
fin función
```

- $T(n) \in \mathrm{O}(nk)$
- El coste espacial es nk.

Conclusiones

- La eficiencia disminuye mucho cuando un problema se divide recursivamente en subproblemas que se solapan.
- Se pueden mantener en memoria los subproblemas resueltos para no repetir cálculos y mejorar la eficiencia.

«Aquellos que no recuerdan su pasado
están condenados a repetirlo»

Principio de optimalidad de Bellman

Principio de optimalidad de Bellman

Enunciado

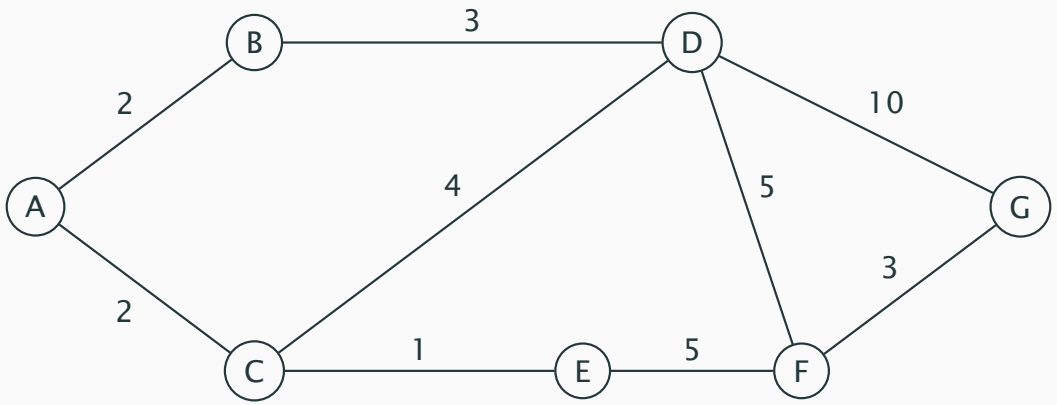
> Una secuencia óptima de decisiones que resuelve un problema debe cumplir la propiedad de que cualquier subsecuencia de decisiones debe ser también óptima respecto al subproblema que resuelve.

En otras palabras

- La solución óptima a cualquier caso no trivial de un problema es una combinación de soluciones óptimas de algunos de los sub-casos.
- Si $d_1, d_2, d_3, \ldots, d_n$ es óptima para $P[1, n]$:
  - Entonces, $d_1, d_2, d_3, \ldots, d_i$ es óptima para $P[1, i]$.
  - Y, $d_{i+1}, \ldots, d_n$ es óptima para $P[i+1, n]$.

Principio de optimalidad de Bellman

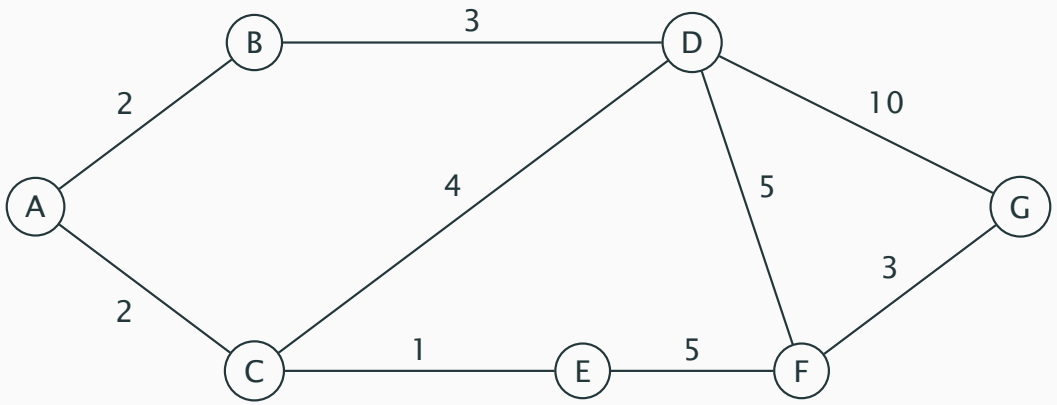
Ejemplo: caminos mínimos



- Camino mínimo de A a G es ACEFG con valor 11.
- Otros caminos de A a G:
  - ACDFG (14).
  - ACDG (16).
  - ABDG (15).
  - ABDFG (13).

Principio de optimalidad de Bellman

Ejemplo: caminos mínimos



- Subcaminos del camino mínimo también son mínimos para el subproblema correspondiente:
  - ACE (3) es el camino mínimo de A a E.
  - EFG (8) es el camino mínimo de E a G.
- La composición de caminos mínimos ABD y DFG no es el camino mínimo de A a G:
  - Camino mínimo de A a D es ABD (5).
  - Camino mínimo de D a G es DFG (8).

Principio de optimalidad de Bellman

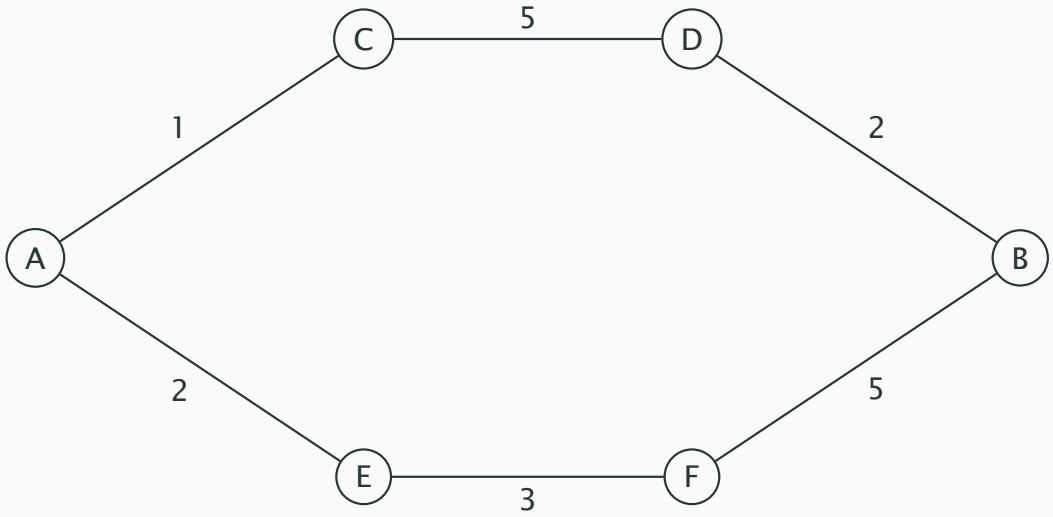
Consideraciones

- La dificultad de aplicar este principio está en que no suele ser evidente cuáles son los sub-casos relevantes para el caso considerado.
- Esto impide usar una aproximación similar a «divide y vencerás», comenzando en el caso original y buscando recursivamente soluciones óptimas para los sub-casos relevantes y solo para estos.
- En su lugar, la programación dinámica resuelve todos los sub-casos para determinar los que realmente son relevantes.
- Entonces, se combinan en una solución óptima para el caso original.

Principio de optimalidad de Bellman

Incumplimiento

- Cálculo del camino simple (sin ciclos) más largo entre dos vértices de un grafo:



- El camino más largo de A a B es AEFB (10), pero el subcamino AEF (5) no es el camino más largo de A a F (es ACDBF (13)).

Características generales

Características de los problemas

- Suelen ser problemas de optimización.
- Deben poder resolverse por etapas.
- Deben poder modelarse mediante una ecuación recurrente.
- Deben cumplir el principio de optimalidad de Bellman.
- Requieren un alto coste computacional (posiblemente exponencial) donde:
  - Subproblemas optimales. La solución óptima a un problema se puede definir en función de soluciones óptimas a subproblemas de tamaño menor, generalmente de forma recursiva.
  - Solapamiento entre subproblemas. Al plantear la solución recursiva, un mismo problema se resuelve más de una vez.

Características de la técnica (1/2)

- Introducida por Richard Bellman en los años 50.
- No tiene nada que ver con la programación (es más bien «planificación»).
- Inspirada en la teoría de control: se obtiene la política óptima para un problema de control con n etapas basándose en una política óptima para un problema similar, pero de $n - 1$ etapas.
- Resuelve un problema por etapas (como *greedy* o *backtracking*).
- Divide el problema en subproblemas (como «divide y vencerás»).

Características de la técnica (2/2)

- Suele ser una técnica ascendente (*bottom-up*):
  - Para obtener la solución, primero calcula las soluciones óptimas a problemas de tamaño pequeño.
  - Usando tales soluciones, encuentra soluciones a problemas de mayor tamaño.
- Para evitar cálculos repetidos, retiene en memoria las soluciones de los sub-problemas (memoización).
- Devuelve la solución óptima (principio de optimalidad de Bellman).

Características generales

Idea general

- Encontrar una formulación recursiva de la solución de un problema mayor en función de soluciones a problemas menores.
- Resolver cada instancia de tamaño menor una única vez y guardarla en una tabla.
- Obtener la solución de la instancia inicial mediante las soluciones almacenadas.

Esquema de diseño

- **Definición de la solución.** Planteamiento de la solución como una sucesión de decisiones y establecimiento de la estructura de la solución.
- **Verificación del principio de optimalidad.** División del problema en subproblemas y demostración de que se cumple el principio de optimalidad.
- **Definición recursiva de la solución óptima.** Cálculo del valor de la solución óptima en función de valores de soluciones para subproblemas de tamaño menor.
  - Determinar el conjunto de subproblemas distintos a resolver (tamaño de la tabla).
  - Identificar los subproblemas con solución trivial.
  - Obtener los valores con un enfoque ascendente y almacenarlos en la tabla. En etapas posteriores, se usarán los valores previamente calculados.
- **Determinación de la solución óptima.** Usando la información calculada previamente, se obtiene la solución óptima.

Análisis de la eficiencia

- Depende del problema, aunque suele ser polinomial.
- En general, al usar una tabla, será nm:
  - n es el tamaño de la tabla.
  - m es el tiempo para rellenar cada casilla.
- Puede dar problemas al necesitar mucha memoria (para rellenar la tabla).

Comparación con «divide y vencerás» (1/2)

- Ambos combinan soluciones de subproblemas.
- Ambos obtienen la solución óptima.
- Dependencia de problemas:
  - «Divide y vencerás» si los subproblemas son independientes.
  - Programación dinámica si los subproblemas se solapan.
- Recursividad e iteración:
  - «Divide y vencerás» usa recurrencias (más tiempo, menos memoria).
  - Programación dinámica intenta evitar recurrencias, usa memoria para la iteración (menos tiempo, más memoria).

Comparación con «divide y vencerás» (2/2)

- Repetición de cálculos:
  - «Divide y vencerás» repite muchos cálculos.
  - Programación dinámica mantiene los resultados en memoria para evitar repetir cálculos.
- Método empleado:
  - «Divide y vencerás» usa un método descendente.
  - Programación dinámica usa un método ascendente (normalmente).

Comparación con *greedy*

- Ambos resuelven el problema por etapas, pero:
  - *Greedy* selecciona un elemento y solo genera una solución, en cada etapa.
  - Programación dinámica selecciona un elemento en cada paso, pero genera múltiples caminos de etapas a seguir, eligiendo la óptima entre estas.
- Optimalidad:
  - *Greedy* no asegura optimalidad (miope).
  - Programación dinámica asegura optimalidad (principio de optimalidad de Bellman).
- Eficiencia:
  - *Greedy* es eficiente en tiempo y memoria.
  - Programación dinámica es eficiente en tiempo pero no en memoria.

Ejemplos de aplicación

El cambio de monedas

Enunciado

Suponiendo que el sistema monetario de un país está formado por monedas de valores $v_1, v_2, \ldots, v_n$, el problema del cambio de monedas consiste en descomponer cualquier cantidad dada M en monedas de ese país usando el menor número posible de monedas.

Ejemplo

- Un artículo de una tienda cuesta 5.37 euros.
- El cliente que lo compra paga con un billete de 10 euros.
- Hay que devolverle 4.63 euros.

¿Cómo se devuelve el cambio con el menor número de monedas?

El cambio de monedas

Algoritmo *greedy*

- Eficiente.
- No era capaz de encontrar el óptimo en todos los casos.

Ejemplo

- Existen monedas de 1, 4 y 6 céntimos. Hay que devolver 8 céntimos.
- El algoritmo *greedy* devuelve:
  - Una moneda de 6 céntimos.
  - Dos monedas de 1 céntimo.
- La solución óptima consiste en dos monedas de 4 céntimos.

Se puede mejorar con programación dinámica

El cambio de monedas

Consideraciones

- Hay monedas de n valores diferentes.
- Las monedas de tipo i tiene un valor de $v_i > 0$ unidades.
- Hay un suministro ilimitado de monedas.
- Al cliente hay que devolverle un cambio de valor igual a M.
- Hay que utilizar el menor número de monedas.

Diseño del algoritmo basado en programación dinámica (1/6)

- **Definición de la solución.** Se puede plantear la solución como una secuencia de decisiones $x_1, x_2, \ldots, x_n$:
  - Cuántas monedas de tipo 1.
  - Cuántas monedas de tipo 2.
  - ...
  - Cuántas monedas de tipo n.

  Sea $C(i, j)$, $1 \leq i \leq n$, $0 \leq j \leq M$, el número mínimo de monedas para obtener la cantidad j restringiéndose a los tipos de moneda desde 1 hasta i. La solución al problema consiste en calcular $C(n, M)$.

- **Verificación del principio de optimalidad.** Si $x_1, x_2, \ldots, x_n$ es óptima para $C(n, M)$, entonces hay que demostrar que $x_1, x_2, \ldots, x_{n-1}$ es óptima para $C(n-1, M - x_n v_n)$.

Diseño del algoritmo basado en programación dinámica (2/6)

Demostración por contradicción. Si no fuese así, entonces existiría una solución $y_1, \ldots, y_{n-1}$ tal que:

$$\sum_{i=1}^{n-1} y_i < \sum_{i=1}^{n-1} x_i \qquad \text{y} \qquad \sum_{i=1}^{n-1} y_i v_i = M - x_n v_n$$

Pero entonces, haciendo $y_n = x_n$ tenemos que:

$$\sum_{i=1}^{n} y_i v_i = M, \qquad \text{luego } y_1, \ldots, y_n \text{ es una solución para } C(n, M).$$

Y, además:

$$\sum_{i=1}^{n} y_i < \sum_{i=1}^{n} x_i, \qquad \text{luego } x_1, \ldots, x_n \text{ no sería óptima.}$$

Diseño del algoritmo basado en programación dinámica (3/6)

- **Definición recursiva de la solución óptima.** En cada paso, existen dos opciones:
  - No incluir ninguna moneda del tipo i. Esto supone que el valor de $C(i, j)$ va a coincidir con el de $C(i-1, j)$. Por tanto:

$$C(i, j) = C(i-1, j)$$

  - Sí incluirla. Al hacerlo, el número de monedas global coincide con el número óptimo de monedas para una cantidad $(j - v_i)$ más esta moneda i que se incluye. Es decir:

$$C(i, j) = 1 + C(i, j - v_i)$$

  Para el cálculo óptimo de $C(i, j)$ se escogerá la solución más favorable (el menor valor de ambas). Además, si no se puede conseguir $C(i, j)$, entonces $C(i, j) = \infty$.

Diseño del algoritmo basado en programación dinámica (4/6)

- Ecuación recurrente:

$$C(i,j) = \begin{cases} \infty, & \text{si } i = 1 \text{ y } 1 \leq j < v_i \\ 0, & \text{si } j = 0 \\ 1 + C(i, j - v_i), & \text{si } i = 1 \text{ y } j \geq v_i \\ C(i-1, j), & \text{si } i > 1 \text{ y } j < v_i \\ \min\left(C(i-1, j), 1 + C(i, j - v_i)\right), & \text{en otro caso} \end{cases}$$

- Se requiere una tabla para almacenar los resultados de los subproblemas.

- Se rellena la tabla de arriba a abajo y de izquierda a derecha.

Diseño del algoritmo basado en programación dinámica (5/6)

```
función cambio_monedas(C[1..n][0..M], v[1..n])
    para todo i = 1, . . . , n hacer
        C[i][0] ← 0
    fin para todo
    para todo i = 1, . . . , n hacer
        para todo j = 1, . . . , M hacer
            si i = 1 && j < v[i] entonces
                C[i][j] ← ∞
            si no si i = 1 entonces
                C[i][j] ← 1 + C[1][j − v[1]]
            si no si j < v[i] entonces
                C[i][j] ← C[i−1][j]
            si no
                C[i][j] ← min(C[i−1][j], 1 + C[i][j−v[i]])
            fin si
        fin para todo
    fin para todo
    devolver C[n][M]
fin función
```

- $T(n) \in \Theta(nM)$

Ejemplo (1/5)

- La cantidad a devolver equivale a ocho unidades ($M = 8$).
- Existen tres tipos de monedas ($n = 3$).
- El valor de cada tipo de moneda es $v_1 = 1$, $v_2 = 4$, y $v_3 = 6$.
- Caso base: $C(i, 0) = 0$.

	0	1	2	3	4	5	6	7	8
$v_1 = 1$	0								
$v_2 = 4$	0								
$v_3 = 6$	0								

Ejemplo (2/5)

- La cantidad a devolver equivale a ocho unidades ($M = 8$).
- Existen tres tipos de monedas ($n = 3$).
- El valor de cada tipo de moneda es $v_1 = 1$, $v_2 = 4$, y $v_3 = 6$.
- $C(1, j) = 1 + C(1, j - 1)$.

	0	1	2	3	4	5	6	7	8
$v_1 = 1$	0	1	2	3	4	5	6	7	8
$v_2 = 4$	0								
$v_3 = 6$	0								

Ejemplo (3/5)

- La cantidad a devolver equivale a ocho unidades ($M = 8$).
- Existen tres tipos de monedas ($n = 3$).
- El valor de cada tipo de moneda es $v_1 = 1$, $v_2 = 4$, y $v_3 = 6$.
- $C(2, j) = \min(C(1, j), 1 + C(2, j - 4))$.

	0	1	2	3	4	5	6	7	8
$v_1 = 1$	0	1	2	3	4	5	6	7	8
$v_2 = 4$	0	1	2	3	1	2	3	4	2
$v_3 = 6$	0								

Ejemplo (4/5)

- La cantidad a devolver equivale a ocho unidades ($M = 8$).
- Existen tres tipos de monedas ($n = 3$).
- El valor de cada tipo de moneda es $v_1 = 1$, $v_2 = 4$, y $v_3 = 6$.
- $C(3, j) = \min(C(2, j), 1 + C(3, j - 6))$.

	0	1	2	3	4	5	6	7	8
$v_1 = 1$	0	1	2	3	4	5	6	7	8
$v_2 = 4$	0	1	2	3	1	2	3	4	2
$v_3 = 6$	0	1	2	3	1	2	1	2	2

Ejemplo (5/5)

- La cantidad a devolver equivale a ocho unidades ($M = 8$).
- Existen tres tipos de monedas ($n = 3$).
- El valor de cada tipo de moneda es $v_1 = 1$, $v_2 = 4$, y $v_3 = 6$.
- La solución óptima es $C(3, 8) = 2$.

	0	1	2	3	4	5	6	7	8
$v_1 = 1$	0	1	2	3	4	5	6	7	8
$v_2 = 4$	0	1	2	3	1	2	3	4	2
$v_3 = 6$	0	1	2	3	1	2	1	2	**2**

Diseño del algoritmo basado en programación dinámica (6/6)

- **Determinación de la solución óptima.** Una vez construida la tabla, es fácil ver qué monedas se necesitan:
  - Se empieza con $C(n, M)$: $i = n$, $j = M$.
  - Si $C(i, j) = C(i-1, j)$, no se escoge una moneda de tipo i y se pasa a analizar $C(i-1, j)$: $i = i - 1$.
  - Si $C(i, j) = 1 + C(i, j - v_i)$, se escoge una moneda de tipo i y se pasa a estudiar $C(i, j - v_i)$: `monedas[i]++`.
  - Se continua hasta que se llega a algún $C(i, 0)$ y ya no quede nada por pagar.

	0	1	2	3	4	5	6	7	8
$v_1 = 1$	0	1	2	3	4	5	6	7	8
$v_2 = 4$	**0**	1	2	3	**1**	2	3	4	**2**
$v_3 = 6$	0	1	2	3	1	2	1	2	**2**

Enunciado

Considérese una mochila capaz de albergar un peso máximo M y n elementos con pesos $p_1, p_2, \ldots, p_3$ y beneficios $b_1, b_2, \ldots, b_n$. Se trata de seleccionar de los n elementos, $x = (x_1, x_2, \ldots, x_n)$, con $x_i \in \{0, 1\}$, aquellos que deberán introducirse en la mochila para maximizar el beneficio. Se buscan los valores $x_1, x_2, \ldots, x_n$, con $x_i \in \{0, 1\}$, que maximizan la siguiente función objetivo:

$$f(x) = \sum_{i=1}^{n} x_i b_i$$

con la restricción:

$$\sum_{i=1}^{n} x_i p_i \leq M$$

Diseño del algoritmo basado en programación dinámica (1/6)

- **Definición de la solución.** Se puede plantear la solución como una secuencia de decisiones $x_1, x_2, \ldots, x_n$:
  - Seleccionar, o no, el objeto 1.
  - Seleccionar, o no, el objeto 2.
  - ...
  - Seleccionar, o no, el objeto n.

  Sea $B(i, p)$, con $1 \leq i \leq n$ y $0 \leq p \leq M$, el beneficio máximo de la mochila con capacidad p cuando consideramos los objetos desde el 1 hasta el i. La solución viene dada por $B(n, M)$.

- **Verificación del principio de optimalidad.** Si $x_1, x_2, \ldots, x_n$ es óptima para $B(n, M)$, entonces hay que demostrar que $x_1, x_2, \ldots, x_{n-1}$ es óptima para:
  - $B(n-1, M)$ si $x_n = 0$.
  - $B(n-1, M-p_i)$ si $x_n = 1$.

Diseño del algoritmo basado en programación dinámica (2/6)

- **Demostración por contradicción.** Caso $x_n = 0$. Si no fuese así, entonces existiría una solución $y_1, \ldots, y_{n-1}$ tal que:

$$\sum_{i=1}^{n-1} y_i b_i > \sum_{i=1}^{n-1} x_i b_i \qquad \text{y} \qquad \sum_{i=1}^{n-1} y_i p_i \leq M$$

Pero entonces, haciendo $y_n = x_n = 0$, tenemos que:

$$\sum_{i=1}^{n} y_i p_i \leq M, \quad \text{luego } y_1, \ldots, y_n \text{ es una solución para } B(n, M)$$

Y, además:

$$\sum_{i=1}^{n} y_i b_i > \sum_{i=1}^{n} x_i b_i, \quad \text{luego } x_1, \ldots, x_n \text{ no sería óptima.}$$

Diseño del algoritmo basado en programación dinámica (3/6)

- **Demostración por contradicción.** Caso $x_n = 1$. Si $x_1, \ldots, x_{n-1}$ no es óptima para $B(n-1, M-p_i)$, entonces existe una solución $y_1, \ldots, y_{n-1}$ tal que:

$$\sum_{i=1}^{n-1} y_i b_i > \sum_{i=1}^{n-1} x_i b_i \qquad \text{y} \qquad \sum_{i=1}^{n-1} y_i p_i \leq M - p_n$$

Pero entonces, haciendo $y_n = x_n = 1$, tenemos que:

$$\sum_{i=1}^{n-1} y_i p_i + p_n \leq M, \quad \text{luego } y_1, \ldots, y_n \text{ es una solución para } B(n, M)$$

Y, además:

$$\sum_{i=1}^{n-1} y_i b_i + b_n > \sum_{i=1}^{n-1} x_i b_i + b_n, \quad \text{luego } x_1, \ldots, x_n \text{ no sería óptima.}$$

Diseño del algoritmo basado en programación dinámica (4/6)

- **Definición recursiva de la solución óptima.** La mejor selección de elementos del conjunto $1, 2, \ldots, i$ para una mochila de tamaño M se puede definir en función de selecciones de elementos de $1, 2, \ldots, i-1$, para mochilas de tamaño menor:
  - O bien es la ganancia para la mejor selección de elementos de $1, 2, \ldots, i-1$ con peso máximo p (no selecciono el objeto i).
  - O bien es la ganancia de la mejor selección de elementos de $1, 2, \ldots, i-1$ con peso máximo $p - p_i$ más la ganancia del elemento i, b_i (sí selecciono el objeto i).

  Ecuación recurrente:

$$B(i,p) = \begin{cases} 0, & \text{si } i = 0 \text{ o } p = 0 \\ B(i-1,p), & \text{si } p_i > p \\ \max(B(i-1,p), B(i-1,p-p_i) + b_i), & \text{en otro caso} \end{cases}$$

  Estos valores se van almacenando en una tabla de tamaño $(n+1)(M+1)$.

Diseño del algoritmo basado en programación dinámica (5/6)

```
función mochila01(B[0..n][0..M], p[1..n]], b[1..n])
    para todo i = 0, . . . , M hacer
        B[0][i] ← 0
    fin para todo
    para todo i = 0, . . . , n hacer
        B[i][0] ← 0
    fin para todo
    para todo i = 1, . . . , n hacer
        para todo j = 1, . . . , M hacer
            si p[i] ≤ j entonces
                B[i][j] ← max(B[i−1][j−p[i]] + b[i], B[i−1][j])
            si no
                B[i][j] ← B[i−1][j]
            fin si
        fin para todo
    fin para todo
    devolver B[n][M]
fin función
```

- $T(n) \in \Theta(nM)$

La mochila 0/1

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- Casos base: $B(0, p) = B(i, 0) = 0$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(1,1) = B(0,1) = 0$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0				
2	0					
3	0					
4	0					

La mochila 0/1

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(1,2) = \max(B(0,2),\ 12 + B(0,2-2)) = \max(0, 12+0) = 12$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12			
2	0					
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(1,3) = \max(B(0,3),\ 12 + B(0,3-2)) = \max(0, 12+0) = 12$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12		
2	0					
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(1,4) = \max(B(0,4),\ 12 + B(0,4-2)) = \max(0, 12+0) = 12$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	
2	0					
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(1,5) = \max(B(0,5),\ 12 + B(0,5-2)) = \max(0, 12+0) = 12$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0					
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(2,1) = \max(B(1,1),\ 10 + B(1,1-1)) = \max(0, 10+0) = 10$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10				
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(2,2) = \max(B(1,2),\ 10 + B(1,2-1)) = \max(12, 10+0) = 12$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12			
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(2,3) = \max(B(1,3),\ 10 + B(1,3-1)) = \max(12, 10+12) = 22$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22		
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(2,4) = \max(B(1,4),\ 10 + B(1,4-1)) = \max(12, 10+12) = 22$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(2,5) = \max(B(1,5),\ 10 + B(1,5-1)) = \max(12, 10+12) = 22$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0					
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(3,1) = B(2,1) = 10$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10				
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(3,2) = B(2,2) = 12$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12			
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(3,3) = \max(B(2,3),\ 20 + B(2,3-3)) = \max(22,\ 20 + 0) = 22$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22		
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(3,4) = \max(B(2,4),\ 20 + B(2,4-3)) = \max(22,\ 20 + 10) = 30$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(3,5) = \max(B(2,5),\ 20 + B(2,5-3)) = \max(22,\ 20 + 12) = 32$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0					

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(4, 1) = B(3, 1) = 10$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10				

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(4, 2) = \max(B(3, 2), 15 + B(3, 2 - 2)) = \max(12, 15 + 0) = 15$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15			

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(4,3) = \max(B(3,3),\ 15 + B(3,3-2)) = \max(12,\ 15 + 10) = 25$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25		

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(4,4) = \max(B(3,4),\ 15 + B(3,4-2)) = \max(30,\ 15 + 12) = 30$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	

Ejemplo

- Supongamos una mochila que puede transportar hasta cinco kilos ($M = 5$) y cuatro objetos ($n = 4$) cuyos pesos y beneficios son:

	1	2	3	4
p_i	2	1	3	2
b_i	12	10	20	15

- $B(4, 5) = \max(B(3, 5), 15 + B(3, 5 - 2)) = \max(32, 15 + 22) = 37$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	**37**

Diseño del algoritmo basado en programación dinámica (6/6)

- **Determinación de la solución óptima.** Una vez construida la tabla, es fácil ver qué objetos se han elegido:
  - Si $B(i, p) = B(i-1, p)$, entonces el objeto i no se selecciona y se pasa a estudiar el objeto $i-1$ para una mochila de capacidad p, $B(i-1, p)$.
  - Si $B(i, p) \neq B(i-1, p)$, entonces el objeto i se selecciona y se pasa a estudiar el objeto $i-1$ para una mochila de capacidad $p - p_i$, $B(i-1, p-p_i)$.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	**12**	12	12	12
2	0	10	12	**22**	22	22
3	0	10	12	**22**	30	32
4	0	10	15	25	30	**37**

Se seleccionan los objetos 4, 2 y 1.

Multiplicación encadenada de matrices

Enunciado

Se quiere calcular la matriz producto M de n matrices dadas, $M = M_1 M_2 \ldots M_n$. Por ser asociativa la multiplicación de matrices, existen muchas formas posibles de realizar esa operación, cada una con un coste asociado en términos del número de multiplicaciones escalares.

Si cada M_i es de dimensión $d_{i-1} \times d_i$ ($1 \leq i \leq n$), multiplicar $M_i M_{i+1}$ requiere $d_{i-1} d_i d_{i+1}$ operaciones. El problema consiste en encontrar el mínimo número de operaciones necesario para calcular el producto M.

Multiplicación encadenada de matrices

Ejemplo

- Sean cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 30×1.
  - M_2 es 1×40.
  - M_3 es 40×10.
  - M_4 es 10×25.
- Existen cinco formas distintas de multiplicar las matrices anteriores, cuyos costes son:

$$
\begin{array}{lrcrcrcr}
((M_1 M_2)M_3)M_4 & = & 30 \cdot 1 \cdot 40 & + & 30 \cdot 40 \cdot 10 & + & 30 \cdot 10 \cdot 25 & = & 20\,700 \\
M_1(M_2(M_3 M_4)) & = & 40 \cdot 10 \cdot 25 & + & 1 \cdot 40 \cdot 25 & + & 30 \cdot 1 \cdot 25 & = & 11\,750 \\
(M_1 M_2)(M_3 M_4) & = & 30 \cdot 1 \cdot 40 & + & 40 \cdot 10 \cdot 25 & + & 30 \cdot 40 \cdot 25 & = & 41\,200 \\
M_1((M_2 M_3)M_4) & = & 1 \cdot 40 \cdot 10 & + & 1 \cdot 10 \cdot 25 & + & 30 \cdot 1 \cdot 25 & = & 1\,400 \\
(M_1(M_2 M_3))M_4 & = & 1 \cdot 40 \cdot 10 & + & 30 \cdot 1 \cdot 10 & + & 30 \cdot 10 \cdot 25 & = & 8\,200
\end{array}
$$

Fuerza bruta (1/2)

- La enumeración de todas las parentizaciones posibles es un método muy ineficiente.
- Sea P_n el número de parentizaciones de una sucesión de n matrices.
- Se puede dividir una sucesión de n matrices en dos (las i primeras y las $n - i$ siguientes) para cualquier $i = 1, 2, \ldots, n - 1$.
- Al parentizar las dos sucesiones resultantes independientemente, se obtiene:

$$P_n = \begin{cases} 1, & \text{si } n = 1 \\ \sum_{i=1}^{n-1} P_i P_{n-i}, & \text{si } n \geq 2 \end{cases}$$

Fuerza bruta (2/2)

- La solución de la ecuación recurrente anterior es la sucesión de los números de Catalan.
- Estos forman una secuencia de números naturales que aparece en varios problemas de conteo que habitualmente son recursivos (por ejemplo, el número de árboles binarios con $n + 1$ hojas).

$$P_n = C_{n-1}$$

- Donde:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} \quad \text{con } n \geq 0$$

- Asintóticamente, los números de Catalan crecen como $\frac{4^n}{n^{3/2}\sqrt{\Pi}}$.
- Por tanto, el método de la fuerza bruta es una pobre estrategia para determinar la parentización óptima de una cadena de matrices.

Diseño del algoritmo basado en programación dinámica (1/6)

- **Definición de la solución.** Se puede plantear la solución como una secuencia de decisiones $x_1, x_2, \ldots, x_n$:
  - Dónde colocar el primer paréntesis.
  - Dónde colocar el segundo paréntesis.
  - ...
  - Dónde colocar el n-ésimo paréntesis.

  Sea $N(i, j)$ el mínimo número de operaciones escalares para multiplicar las matrices $M_i \ldots M_j$. La solución que se busca es $N(1, n)$.

Diseño del algoritmo basado en programación dinámica (2/6)

- **Verificación del principio de optimalidad.** La solución óptima se puede definir en términos de soluciones óptimas a problemas de tamaño menor:
  - Necesariamente tiene que haber una multiplicación final (la de mayor nivel) en el cálculo de la solución óptima.
  - Si $(M_1(M_2M_3))M_4$ es óptima para $M_1M_2M_3M_4$, entonces $(M_1(M_2M_3))$ es óptima para $M_1M_2M_3$.
  - Si hubiera una solución mejor para el subproblema, se podría haber usado en lugar de la anterior, lo que sería una contradicción sobre la optimalidad de $(M_1(M_2M_3))M_4$.

Diseño del algoritmo basado en programación dinámica (3/6)

- **Definición recursiva de la solución óptima.** Sea $d_{i-1} \times d_i$ la dimensión de la matriz M_i. Se quiere multiplicar $(M_1 M_2 \ldots M_i M_{i+1} M_{i+2} \ldots M_n)$.

  Supongamos que parentizamos en i:

  $$(M_1 M_2 \ldots M_i)(M_{i+1} M_{i+2} \ldots M_n)$$

  Si $N(i, j)$ el número de operaciones necesarias para multiplicar $M_i M_{i+1} \ldots M_j$, entonces:

  $$N(1, n) = N(1, i) + N(i + 1, n) + d_0 d_i d_n$$

Diseño del algoritmo basado en programación dinámica (4/6)

- Ecuación recurrente:

$$N(i,j) = \begin{cases} 0, & \text{si } i = j \\ \min_{i \le k < j} (N(i,k) + N(k+i,j) + d_{i-1} d_k d_j), & \text{si } 1 \le i < j \le n \end{cases}$$

- A partir de la definición recursiva se puede ver cómo se produce el solapamiento de los subproblemas.

- Primero se solucionan los subproblemas triviales (tamaño 0) para en cada paso ir resolviendo subproblemas de tamaño $1, 2, 3, \ldots$

Diseño del algoritmo basado en programación dinámica (5/6)

```
función multiplicación_encadenada(N[1..n][1..n], d[0..n])
    para todo i = 1, . . . , n hacer
        N[i][i] ← 0
    fin para todo
    para todo l = 2, . . . , n hacer
        para todo i = 1, . . . , n−1 hacer
            j ← i + l
            N[i][j] ← ∞
            para todo k = i. . . j−1 hacer
                aux ← N[i][k] + N[k + 1][j] + d[i]*d[k + 1]*d[j + 1]
                si aux < N[i][j] entonces
                    N[i][j] ← aux
                fin si
            fin para todo
        fin para todo
    fin para todo
    devolver N[1][n]
fin función
```

- $T(n) \in \Theta(n^3)$

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- Caso base: $N(i, i) = 0$.

	1	2	3	4
1	0			
2		0		
3			0	
4				0

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- $N(1,2) = N(1,1) + N(2,2) + d_0 d_1 d_2 = 0 + 0 + 10\,000 = 10\,000$.

	1	2	3	4
1	0	10 000		
2		0		
3			0	
4				0

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- $N(2,3) = N(2,2) + N(3,3) + d_1 d_2 d_3 = 0 + 0 + 1\,000 = 1\,000$.

	1	2	3	4
1	0	10 000		
2		0	1 000	
3			0	
4				0

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- $N(3,4) = N(3,3) + N(4,4) + d_2 d_3 d_4 = 0 + 0 + 5\,000 = 5\,000$.

	1	2	3	4
1	0	10 000		
2		0	1 000	
3			0	5 000
4				0

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- $N(1,3) = \min(N(1,1) + N(2,3) + d_0 d_1 d_3, N(1,2) + N(3,3) + d_0 d_2 d_3) = \min(0 + 1\,000 + 200, 10\,000 + 0 + 500) = 1\,200$.

	1	2	3	4
1	0	10 000	1 200	
2		0	1 000	
3			0	5 000
4				0

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- $N(2,4) = \min(N(2,2) + N(3,4) + d_1 d_2 d_4, N(2,3) + N(4,4) + d_1 d_3 d_4) = \min(0 + 5\,000 + 100\,000, 1\,000 + 0 + 2\,000) = 3\,000$.

	1	2	3	4
1	0	10 000	1 200	
2		0	1 000	3 000
3			0	5 000
4				0

Multiplicación encadenada de matrices

Ejemplo

- Supongamos cuatro matrices, M_1, M_2, M_3 y M_4, cuyos órdenes son:
  - M_1 es 10×20.
  - M_2 es 20×50.
  - M_3 es 50×1.
  - M_4 es 1×200.
- $N(1,4) = \min(N(1,1) + N(2,4) + d_0 d_1 d_4, N(1,2) + N(3,4) + d_0 d_2 d_4, N(1,3) + N(4,4) + d_0 d_3 d_4) = 2\,200$.

	1	2	3	4
1	0	10 000	1 200	2 200
2		0	1 000	3 000
3			0	5 000
4				0

Diseño del algoritmo basado en programación dinámica (6/6)

- **Determinación de la solución óptima.** Usar una matriz auxiliar, `Mejor_i(1..n,1..n)`, en la que se almacene el índice donde se alcanzó el mínimo (mejor valor de i) para cada subproblema.

	1	2	3	4
1	—	1	1	3
2		—	2	3
3			—	3
4				—

- `Mejor_i(1,4) = 3`, luego los subproblemas son $M_{1..3}$ y $M_{4..4}$. Es decir, $(M_1 M_2 M_3)M_4$.

- `Mejor_i(1,3) = 1`, luego tenemos $M_{1..1}$ y $M_{2..3}$. Es decir, $M_1(M_2 M_3)$.

- La parametrización óptima es $((M_1(M_2 M_3))M_4)$.

Caminos mínimos

Enunciado

Sea $G = (V, A, c)$ un grafo dirigido y ponderado, donde $c : A \to \mathbb{R}^+$ es la función que asigna a cada arco $\langle u, v \rangle$ un coste $c(u, v) > 0$. Se desea obtener el camino más corto que une cada par de vértices del grafo.

Posibles soluciones

- Fuerza bruta (orden exponencial).
- Aplicar el algoritmo de Dijkstra para cada vértice.
- Algoritmo de Floyd (basado en programación dinámica).

Diseño del algoritmo basado en programación dinámica (1/6)

- **Definición de la solución.** Se puede plantear la solución como una secuencia de decisiones $x_1, x_2, \ldots, x_n$:
  - Cuál es el primer vértice del camino.
  - Cuál es el segundo vértice del camino.
  - ...
  - Cuál es el n-ésimo vértice del camino.

  Sea $D_k(i, j)$ el coste del camino más corto que va del vértice i al vértice j usando solo los k primeros vértices del grafo, $\{1, 2, \ldots, k\}$, como vértices intermedios. Si $k = 0$, no hay vértices intermedios. La solución al problema consiste en calcular $D_n(i, j)$, siendo n el número de vértices del grafo.

Diseño del algoritmo basado en programación dinámica (2/6)

- **Verificación del principio de optimalidad.**
  - Si el camino de coste mínimo del vértice i al vértice j pasa por el vértice k, entonces los caminos de coste mínimo de i a k y de k a j son también caminos de coste mínimo.
  - Si no lo fueran, entonces se encontraría un camino de i a j mejor que el mínimo.
  - Por tanto, se cumple el principio de optimalidad de Bellman.

Diseño del algoritmo basado en programación dinámica (3/6)

- **Definición recursiva de la solución óptima.** En cada etapa, existen dos posibles opciones:
  - Si el camino de coste mínimo del vértice i al vértice j usando solo los k primeros vértices no pasa por el vértice k:

$$D_k(i,j) = D_{k-1}(i,j)$$

  - Si el camino de coste mínimo del vértice i al vértice j usando solo los k primeros vértices sí pasa por el vértice k:

$$D_k(i,j) = D_{k-1}(i,k) + D_{k-1}(k,j)$$

Por tanto, la ecuación recurrente es:

$$D_k(i,j) = \begin{cases} c_{ij}, & \text{si } k = 0 \\ \min\left(D_{k-1}(i,j), D_{k-1}(i,k) + D_{k-1}(k,j)\right), & \text{en otro caso} \end{cases}$$

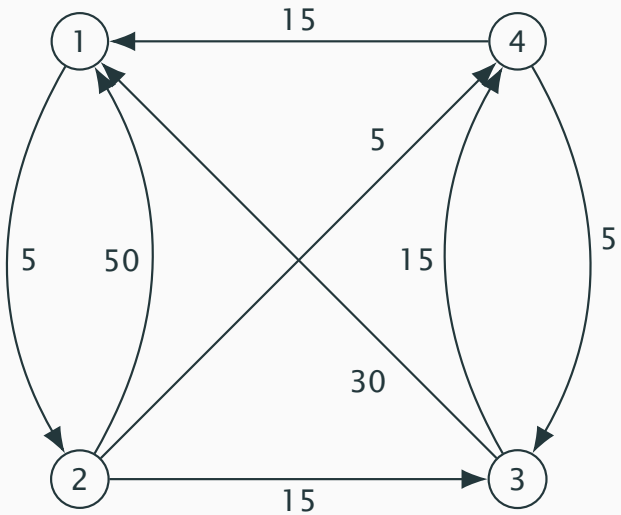
Donde $C = (c_{ij})$ es la matriz de adyacencia del grafo, con $c_{ij} = \infty$ si no hay arco del vértice i al vértice j.

Diseño del algoritmo basado en programación dinámica (4/6)

```
procedimiento floyd(C[1..n][1..n], D[1..n][1..n)
    para todo i = 1, . . . , n hacer
        para todo j = 1, . . . , n hacer
            D[i][j] ← C[i][j]
        fin para todo
    fin para todo
    para todo k = 1, . . . , n hacer
        para todo i = 1, . . . , n hacer
            para todo j = 1, . . . , n hacer
                si D[i][k] + D[k][j] < D[i][j] entonces
                    D[i][j] ← D[i][k] + D[k][j]
                fin si
            fin para todo
        fin para todo
    fin para todo
fin procedimiento
```

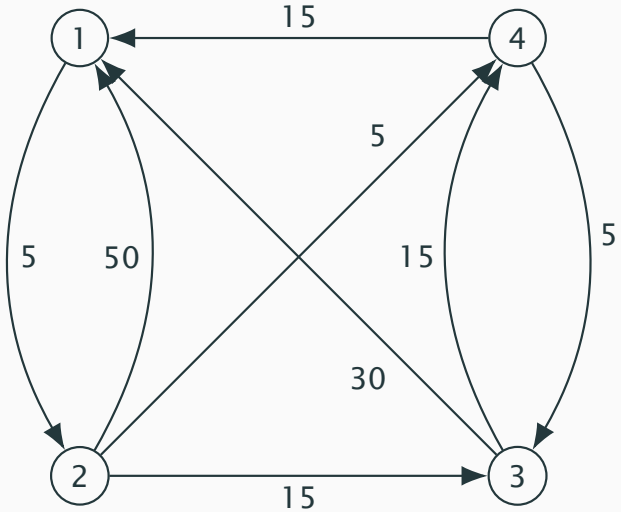
- $T(n) \in \Theta(n^3)$

Ejemplo



$$D_0 = C = \begin{bmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{bmatrix}$$

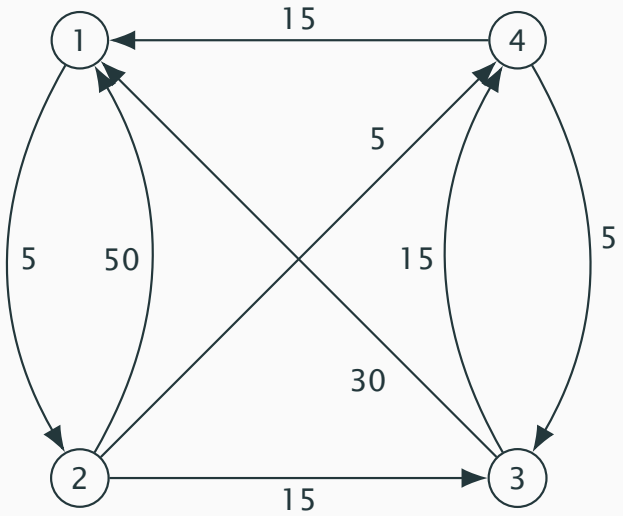
Ejemplo



- $D_1(3,2) = \min(D_0(3,2),\ D_0(3,1) + D_0(1,2)) = 35$
- $D_1(4,2) = \min(D_0(4,2),\ D_0(4,1) + D_0(1,2)) = 20$
- $D_1(4,3) = \min(D_0(4,3),\ D_0(4,1) + D_0(1,3)) = 5$

$$D_1 = \begin{bmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix}$$

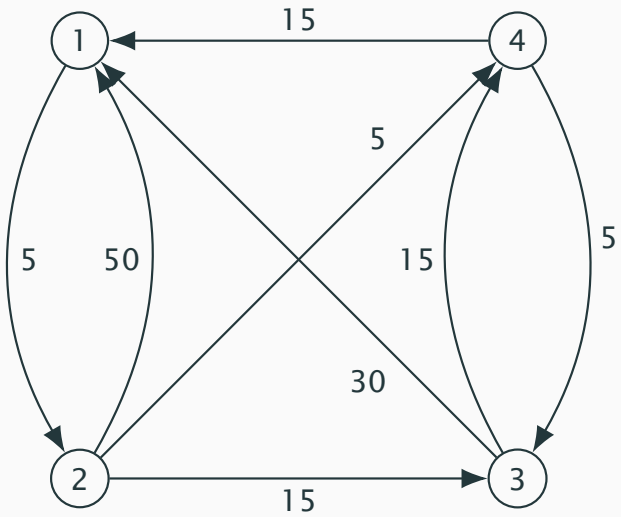
Ejemplo



- $D_2(1,3) = \min(D_1(1,3),\ D_1(1,2) + D_1(2,3)) = 20$
- $D_2(1,4) = \min(D_1(1,4),\ D_1(1,2) + D_1(2,4)) = 10$
- $D_2(4,3) = \min(D_1(4,3),\ D_1(4,2) + D_1(2,3)) = 5$

$$D_2 = \begin{bmatrix} 0 & 5 & 20 & 10 \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix}$$

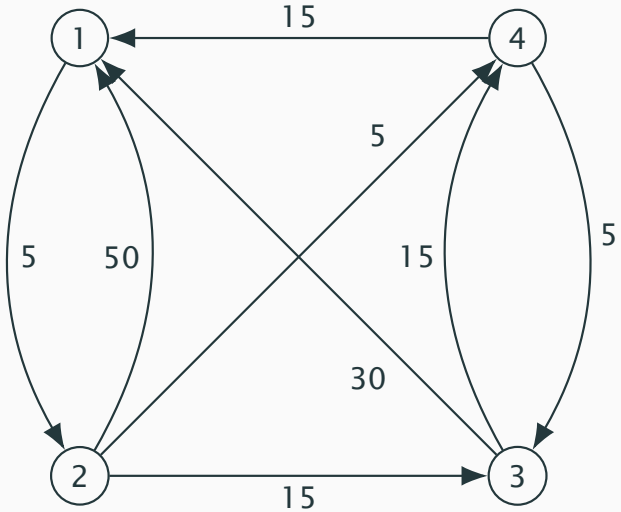
Ejemplo



- $D_3(2,1) = \min(D_2(2,1),\ D_2(2,3) + D_2(3,1)) = 45$

$$D_3 = \begin{bmatrix} 0 & 5 & 20 & 10 \\ 45 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix}$$

Ejemplo



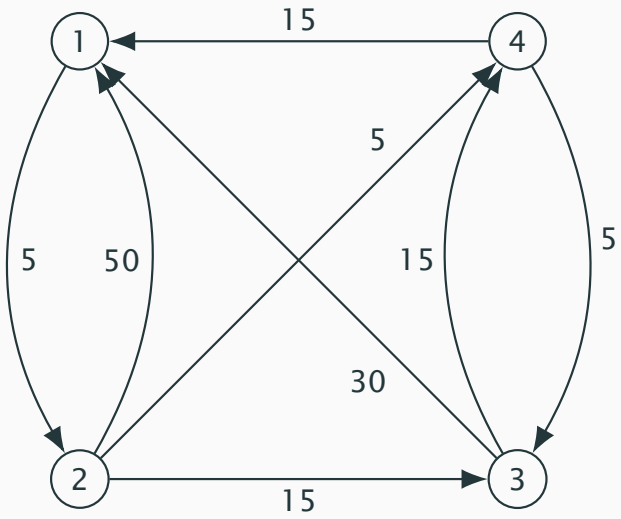
- $D_4(1,3) = \min(D_3(1,3),\ D_3(1,4) + D_3(4,3)) = 15$
- $D_4(2,1) = \min(D_3(2,1),\ D_3(2,4) + D_3(4,1)) = 20$
- $D_4(2,3) = \min(D_3(2,3),\ D_3(2,4) + D_3(4,3)) = 15$

$$D_4 = \begin{bmatrix} 0 & 5 & 20 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix}$$

Diseño del algoritmo basado en programación dinámica (5/6)

- **Determinación de la solución óptima.** Si además de conocer el coste del camino mínimo se quiere conocer el camino en sí, se emplea otra matriz P de forma que:
  - Si $D(i,j) + D(k,j) < D(i,j)$, entonces $P(i,j) = k$.
  - Si al terminar $P(i,j) = 0$, el camino es el directo de i a j.
  - Si $P(i,j) = k$, entonces el camino pasa por k. Se analizan $P(i,k)$ y $P(k,j)$.

Diseño del algoritmo basado en programación dinámica (6/6)



- Camino de 1 a 3 es $P(1,3) = 4$.
- Camino de 1 a 4 y de 4 a 3: $P(1,4) = 2$ y $P(4,3) = 0$.
- Camino de 1 a 2 y de 2 a 4: $P(1,2) = 0$ y $P(2,4) = 0$.
- Camino mínimo de 1 a 3 es: 1-2-4-3.

$$P = \begin{bmatrix} 0 & 0 & 4 & 2 \\ 4 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Bibliografía

Bibliografía

Aclaración

- El contenido de las diapositivas es esquemático y representa un apoyo para las clases teóricas.

- Se recomienda completar los contenidos del tema 4 con apuntes propios tomados en clase y con la bibliografía principal de la asignatura.

Por ejemplo


G. Brassard and P. Bratley.
***Fundamentals of Algorithmics*.**
Prentice Hall, Englewwod Cliffs, New Jersey, 1996.


T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
***Introduction to Algorithms*.**
The MIT Press, Cambridge, MA, 2022.


J. L. Verdegay.
***Lecciones de Algorítmica*.**
Editorial Técnica AVICAM, 2017.